

8 Puzzle Problem

```
import java.util.*;

public class AStar8Puzzle {

    static class Node {
        int[][] state;
        int x, y;      // blank tile position
        int g, h, f;    // costs
        Node parent;

        Node(int[][] state, int x, int y, int g, Node parent) {
            this.state = state;
            this.x = x;
            this.y = y;
            this.g = g;
            this.parent = parent;
            this.h = calculateHeuristic(state);
            this.f = this.g + this.h;
        }

        // Misplaced tiles heuristic
        static int calculateHeuristic(int[][] state) {
            int[][] goal = {
                {1,2,3},
                {4,5,6},
                {7,8,0}
            };
        }
    }
}
```

```

    int count = 0;
    for(int i=0;i<3;i++) {
        for(int j=0;j<3;j++) {
            if(state[i][j] != 0 && state[i][j] != goal[i][j])
                count++;
        }
    }
    return count;
}

```

```

static void printState(int[][] state) {
    for(int i=0;i<3;i++) {
        for(int j=0;j<3;j++)
            System.out.print(state[i][j] + " ");
        System.out.println();
    }
    System.out.println();
}

```

```

static boolean isGoal(int[][] state) {
    int[][] goal = {
        {1,2,3},
        {4,5,6},
        {7,8,0}
    };
    return Arrays.deepEquals(state, goal);
}

```

```

static int[][] copy(int[][] state) {

```

```

int[][] newState = new int[3][3];
for(int i=0;i<3;i++)
    newState[i] = state[i].clone();
return newState;
}

```

```

static void solve(int[][] start, int x, int y) {

```

```

    PriorityQueue<Node> open =
        new PriorityQueue<>(Comparator.comparingInt(n -> n.f));

```

```

    Set<String> closed = new HashSet<>();

```

```

    open.add(new Node(start, x, y, 0, null));

```

```

    int[] dx = {1,-1,0,0};

```

```

    int[] dy = {0,0,1,-1};

```

```

    while(!open.isEmpty()) {

```

```

        Node current = open.poll();

```

```

        if(isGoal(current.state)) {

```

```

            System.out.println("Goal Reached!\nSolution Path:");

```

```

            printPath(current);

```

```

            return;

```

```

        }

```

```

        closed.add(Arrays.deepToString(current.state));

```

```

        for(int i=0;i<4;i++) {

```

```

int nx = current.x + dx[i];
int ny = current.y + dy[i];

if(nx>=0 && nx<3 && ny>=0 && ny<3) {
    int[][] newState = copy(current.state);

    // swap blank
    newState[current.x][current.y] =
        newState[nx][ny];
    newState[nx][ny] = 0;

    if(closed.contains(
        Arrays.deepToString(newState)))
        continue;

    open.add(new Node(newState,
        nx, ny,
        current.g + 1,
        current));
}
}

System.out.println("No Solution Found");
}

static void printPath(Node node) {
    if(node == null) return;
    printPath(node.parent);
    printState(node.state);
}

```

```

    }

    public static void main(String[] args) {

        int[][] start = {
            {1,2,3},
            {4,0,6},
            {7,5,8}
        };

        solve(start, 1, 1);
    }
}

```

Output

Goal Reached!

Solution Path:

1 2 3

4 0 6

7 5 8

1 2 3

4 5 6

7 0 8

1 2 3

4 5 6

7 8 0